A04 Trees & Maps Assignment

#algorithmic_thinking #java #trees #maps

- ***Name:*** Zoe McFife
- ***Legal Name and Student NR:*** Bunea, S2510238021
- ***Time Spent***: 06:42:14

---

1 Ye Old Item Shoppe — A BST-Powered Shop

1.1 `remove(T value)` on BinarySearchTree

BST remove Implementation.

```
public void remove(T value)
{
    if (!contains(value))
    {
        return;
    }

    root = remove(root, value);
}

private TreeNode<T> remove(TreeNode<T> node, T value)
{
    int cmp = value.compareTo(node.value);

    if (cmp < 0)
    {
        node.left = remove(node.left, value);
    }
    else if (cmp > 0)
    {
        node.right = remove(node.right, value);
    }
    else
    {
        if (node.left == null)
        {
            return node.right;
        }

        if (node.right == null)
```

```
        {
            return node.left;
        }

        // Two Children
        node.value = findSmallest(node.right).value;
        node.right = remove(node.right, node.value);
    }

    return node;
}

private TreeNode<T> findSmallest(TreeNode<T> node)
{
    while (node.left != null)
        node = node.left;

    return node;
}
```

For cases `no children` and `only one child`, I used two if clauses.

If either right or left node is null, return the other node. If both are null, null gets returned regardless. The parent node gets set to null or the child node.

```
if (node.left == null)
{
    return node.right;
}


if (node.right == null)
{
    return node.left;
}
```

If the node has two child nodes, I set its value to the smallest value in the right subtree. I created a helper method to find the smallest value.

```
    // Two Children
    node.value = findSmallest(node.right).value;
    node.right = remove(node.right, node.value);
```

1.2 `rangeSearch(T min, T max)` on BinarySearchTree

I use `inorder()` to first create a sorted list of all values. I filter that list using `compareTo` and return it.

```
public List<T> rangeSearch(T min, T max)
{
    if (min.compareTo(max) ⩾ 0)
    {
        throw new IllegalArgumentException("Invalid range; Min: " + min +
", Max: " + max + "; max must be larger than min!");
    }

    return inorder().stream()
            .filter(v → v.compareTo(min) ⩾ 0 && v.compareTo(max) ⩽ 0)
            .toList();
}
```

1.3 Shop Simulation

1.3.1 `ShopItem`

```
public class ShopItem extends Item implements Comparable<ShopItem>
{
    private final int price;
    private final int offense;
    private final int defense;
    private int stock;

    public ShopItem(String name, String type, int weight, int value, int
price, int offense, int defense, int stock)
    {
        super(name, type, weight, value);
        this.price = price;
        this.offense = offense;
        this.defense = defense;
        setStock(stock);
    }

    public void increaseStock()
    {
        setStock(getStock() + 1);
    }

    public void decreaseStock()
    {
        setStock(getStock() - 1);
    }

    private void setStock(int stock)
```

```
    {
        if (stock < 0)
        {
            this.stock = 0;
            return;
        }

        this.stock = stock;
    }

    public int getStock()
    {
        return stock;
    }

    public int getPrice()
    {
        return price;
    }

    public int getOffense()
    {
        return offense;
    }

    public int getDefense()
    {
        return defense;
    }

    @Override
    public int compareTo(ShopItem o)
    {
        // Price
        int cmp = Integer.compare(this.price, o.price);

        if (cmp != 0) return cmp;

        // if same price, compare name
        cmp = this.getName().compareTo(o.getName());
        if (cmp != 0) return cmp;

        // same name, compare type
        return this.getType().compareTo(o.getType());
    }
}
```

For comparing, I used the name and type as secondary comparisons, in case the price is equal.

1.3.2 Customer

```
public class Customer
{
    private final String name;
    private final int budget;
    private final boolean prefersOffense;

    public Customer(String name, int budget, boolean prefersOffense)
    {
        this.name = name;
        this.budget = budget;
        this.prefersOffense = prefersOffense;
    }

    public String getName()
    {
        return this.name;
    }

    public int getBudget()
    {
        return this.budget;
    }

    public boolean prefersOffense()
    {
        return this.prefersOffense;
    }

    @Override
    public String toString()
    {
        return "Customer: " + this.name + " Budget: " + this.budget + "
Prefers Offense?: " + this.prefersOffense;
    }
}
```

1.3.3 Shop

Simple shop. Items get updated via delete / insertions.

```
public class Shop extends BinarySearchTree<ShopItem>
{
    public void restock(ShopItem item)
    {
        if (contains(item))
        {
```

```
            remove(item);

            item.increaseStock();

            insert(item);

            return;
        }

        insert(item);
    }

    public void bought(ShopItem item)
    {
        if (contains(item))
        {
            remove(item);

            item.decreaseStock();

            if (item.getStock() != 0)
            {
                insert(item);
            }
            else
            {
                IO.println(item.getName() + " is out of stock!");
            }
        }
    }

    public ShopItem processSale(Customer customer)
    {
        ShopItem item = getBestShopItem(customer);

        if (item == null) return null;

        bought(item);

        IO.println(customer.getName() + " bought " + item.getName());
        IO.println("Current Stock: " + this);

        return item;
    }

    private ShopItem getBestShopItem(Customer customer)
    {
        ShopItem freeShopItem = new ShopItem("Free", "Free", 0, 0, 0, 0,
0, 0);
        ShopItem mostExpensiveShopItem = new ShopItem("Most Expensive",
```

```
"Most Expensive", Integer.MAX_VALUE, Integer.MAX_VALUE,
customer.getBudget(), Integer.MAX_VALUE, Integer.MAX_VALUE,
Integer.MAX_VALUE);

        List<ShopItem> affordableItems = rangeSearch(freeShopItem,
mostExpensiveShopItem);

        if (affordableItems.isEmpty()) return null;

        if (affordableItems.size() == 1) return
affordableItems.getFirst();

        Optional<ShopItem> bestItem;

        if (customer.prefersOffense())
        {
            bestItem =
affordableItems.stream().max(Comparator.comparingInt(ShopItem::getOffense)
);
        }
        else
        {
            bestItem =
affordableItems.stream().max(Comparator.comparingInt(ShopItem::getDefense)
);
        }

        return bestItem.orElse(null);
    }
}
```

1.3.4 Shop Demo

I set up a simple loop that process the sale of each customer (twice).

```
public static void main()
{
    setCustomers();
    stockShop();

    customers.forEach(shop::processSale);
    customers.forEach(shop::processSale);
}
```

```
public static void setCustomers()
{
    /*Auto generated random customers*/
```

```
    customers.addAll(new ArrayList<>(List.of(
            // budget in aUEC
            new Customer("Jax Renly",       1500, true),
            new Customer("Vera Osei",        3000, false),
            new Customer("Drex Korvath",     7000, true),
            new Customer("Sable Tannis",     500,  false),
            new Customer("Admiral Rix Vane", 9999, false)
    )));
}

public static void stockShop()
{
    /*Auto generated random items*/

    List<ShopItem> items = new ArrayList<>(List.of(
            // Weapons
            new ShopItem("Klaus & Werner P8-SC",  "SMG",        2,  420,
800, 18,  3, 2),
            new ShopItem("Kastak Arms Devastator", "Shotgun",    5,  900,
1400, 28,  1, 1),
            new ShopItem("Gemini LH86 Pistol",    "Pistol",     1,  180,
350, 10,  2, 1),
            // Armor
            new ShopItem("Lightstrike V Helmet",  "Helmet",     3,  500,
900,  0, 20, 2),
            new ShopItem("Pyro RGD Chest Plate",  "Armor",      8, 1100,
1800,  0, 35, 1),
            new ShopItem("Novikov EVA Suit",       "Space Suit", 6,  800,
1300,  1, 28, 1),
            // Ship Components
            new ShopItem("M7A Laser Cannon",      "Ship Weapon",20, 4000,
6500, 50,  0, 1),
            new ShopItem("AllMax S2 Shield Gen",  "Shield",     15, 3200,
5000,  0, 60, 2),
            // Gear / Consumables
            new ShopItem("Stims MedPen",          "Medical",    0,   80,
120,  0,  5,2),
            new ShopItem("BRT4 Breacher Grenade", "Explosive",  1,  300,
500, 22,  0, 3)
    ));
    items.forEach(shop::restock);
}
```

Output:

```
Kastak Arms Devastator is out of stock!

Jax Renly bought Kastak Arms Devastator
```

```
Current Stock: [Stims MedPen [Medical, w=0, v=80], Gemini LH86 Pistol
[Pistol, w=1, v=180], BRT4 Breacher Grenade [Explosive, w=1, v=300], Klaus
& Werner P8-SC [SMG, w=2, v=420], Lightstrike V Helmet [Helmet, w=3,
v=500], Novikov EVA Suit [Space Suit, w=6, v=800], Pyro RGD Chest Plate
[Armor, w=8, v=1100], AllMax S2 Shield Gen [Shield, w=15, v=3200], M7A
Laser Cannon [Ship Weapon, w=20, v=4000]]

Pyro RGD Chest Plate is out of stock!

Vera Osei bought Pyro RGD Chest Plate

Current Stock: [Stims MedPen [Medical, w=0, v=80], Gemini LH86 Pistol
[Pistol, w=1, v=180], BRT4 Breacher Grenade [Explosive, w=1, v=300], Klaus
& Werner P8-SC [SMG, w=2, v=420], Lightstrike V Helmet [Helmet, w=3,
v=500], Novikov EVA Suit [Space Suit, w=6, v=800], AllMax S2 Shield Gen
[Shield, w=15, v=3200], M7A Laser Cannon [Ship Weapon, w=20, v=4000]]

M7A Laser Cannon is out of stock!

Drex Korvath bought M7A Laser Cannon

Current Stock: [Stims MedPen [Medical, w=0, v=80], Gemini LH86 Pistol
[Pistol, w=1, v=180], BRT4 Breacher Grenade [Explosive, w=1, v=300], Klaus
& Werner P8-SC [SMG, w=2, v=420], Lightstrike V Helmet [Helmet, w=3,
v=500], Novikov EVA Suit [Space Suit, w=6, v=800], AllMax S2 Shield Gen
[Shield, w=15, v=3200]]

Sable Tannis bought Stims MedPen

Current Stock: [Stims MedPen [Medical, w=0, v=80], Gemini LH86 Pistol
[Pistol, w=1, v=180], BRT4 Breacher Grenade [Explosive, w=1, v=300], Klaus
& Werner P8-SC [SMG, w=2, v=420], Lightstrike V Helmet [Helmet, w=3,
v=500], Novikov EVA Suit [Space Suit, w=6, v=800], AllMax S2 Shield Gen
[Shield, w=15, v=3200]]
Admiral Rix Vane bought AllMax S2 Shield Gen
Current Stock: [Stims MedPen [Medical, w=0, v=80], Gemini LH86 Pistol
[Pistol, w=1, v=180], BRT4 Breacher Grenade [Explosive, w=1, v=300], Klaus
& Werner P8-SC [SMG, w=2, v=420], Lightstrike V Helmet [Helmet, w=3,
v=500], Novikov EVA Suit [Space Suit, w=6, v=800], AllMax S2 Shield Gen
[Shield, w=15, v=3200]]

Jax Renly bought BRT4 Breacher Grenade

Current Stock: [Stims MedPen [Medical, w=0, v=80], Gemini LH86 Pistol
[Pistol, w=1, v=180], BRT4 Breacher Grenade [Explosive, w=1, v=300], Klaus
& Werner P8-SC [SMG, w=2, v=420], Lightstrike V Helmet [Helmet, w=3,
v=500], Novikov EVA Suit [Space Suit, w=6, v=800], AllMax S2 Shield Gen
[Shield, w=15, v=3200]]

Novikov EVA Suit is out of stock!
```

```
Vera Osei bought Novikov EVA Suit

Current Stock: [Stims MedPen [Medical, w=0, v=80], Gemini LH86 Pistol
[Pistol, w=1, v=180], BRT4 Breacher Grenade [Explosive, w=1, v=300], Klaus
& Werner P8-SC [SMG, w=2, v=420], Lightstrike V Helmet [Helmet, w=3,
v=500], AllMax S2 Shield Gen [Shield, w=15, v=3200]]

Drex Korvath bought BRT4 Breacher Grenade

Current Stock: [Stims MedPen [Medical, w=0, v=80], Gemini LH86 Pistol
[Pistol, w=1, v=180], BRT4 Breacher Grenade [Explosive, w=1, v=300], Klaus
& Werner P8-SC [SMG, w=2, v=420], Lightstrike V Helmet [Helmet, w=3,
v=500], AllMax S2 Shield Gen [Shield, w=15, v=3200]]

Stims MedPen is out of stock!

Sable Tannis bought Stims MedPen

Current Stock: [Gemini LH86 Pistol [Pistol, w=1, v=180], BRT4 Breacher
Grenade [Explosive, w=1, v=300], Klaus & Werner P8-SC [SMG, w=2, v=420],
Lightstrike V Helmet [Helmet, w=3, v=500], AllMax S2 Shield Gen [Shield,
w=15, v=3200]]

AllMax S2 Shield Gen is out of stock!

Admiral Rix Vane bought AllMax S2 Shield Gen

Current Stock: [Gemini LH86 Pistol [Pistol, w=1, v=180], BRT4 Breacher
Grenade [Explosive, w=1, v=300], Klaus & Werner P8-SC [SMG, w=2, v=420],
Lightstrike V Helmet [Helmet, w=3, v=500]]
```

---

2 Dungeon Run Log — Loops vs. Streams

I have to say, the instructions for this part are kinda all over the place and don't make much sense, to me at least…

2.1 `DungeonRun` and random generator

2.1.1 `DungeonPlayer`

`DungeonPlayer` represents a player running through a dungeon and saves the performance.

```
public class DungeonPlayer
{
    private final String name;
    private int gold = 0;
    private int roomsVisited = 0;
    private int health;
    private final List<Item> items = new ArrayList<>();

    private boolean isDead = false;

    public DungeonPlayer(String name, int health)
    {
        this.name = name;
        this.health = health;
    }

    // damage, heal and other getter setter methods are below

}
```

2.1.2 `DungeonRoom`

A `DungeonRoom` extends `Room` and has values that will affect the `DungeonPlayer`. Each room has items, gold and some amount of damage it will give to a player and how much it will heal a player.

```
public class DungeonRoom extends Room
{
    private final List<Item> items = new ArrayList<>();
    private final int roomDamage;
    private final int roomHealth;
    private final int gold;

    public DungeonRoom(String name, String description, List<Item> items,
int roomDamage, int roomHealth, int gold)
    {
        super(name, description);
        this.items.addAll(items);
        this.roomDamage = roomDamage;
        this.roomHealth = roomHealth;
        this.gold = gold;
    }

    // getter setters ignored

}
```

2.1.3 `DungeonGenerator`

`DungeonGenerator` creates a random `DungeonPlayer` and procedurally creates a certain amount of `DungeonRoom`s. It uses a seed, so all dungeons can be deterministically generated.

```
public class DungeonGenerator
{
    private final List<String> names = new ArrayList<>();
    private final List<Item> items = new ArrayList<>();

    private final long seed;
    private final Random random;
    private final int roomCount;

    private final DungeonPlayer dungeonPlayer;
    private final List<DungeonRoom> dungeonRooms;

    public DungeonGenerator(int roomCount, long seed)
    {
        createNames();
        createItems();

        this.roomCount = roomCount;
        this.seed = seed;
        this.random = new Random(seed);

        dungeonPlayer = createDungeonPlayer();
        dungeonRooms = createDungeonRooms();
    }

    public DungeonPlayer getDungeonPlayer()
    {
        return dungeonPlayer;
    }

    public DungeonRoom getDungeonRoom(int index)
    {
        return dungeonRooms.get(index);
    }

    public List<DungeonRoom> getDungeonRooms()
    {
        return dungeonRooms;
    }

    private DungeonPlayer createDungeonPlayer()
    {
        return new DungeonPlayer(getRandomName(), random.nextInt(50,
```

```
150));
    }

    private List<DungeonRoom> createDungeonRooms()
    {
        List<DungeonRoom> dungeonRooms = new ArrayList<>();

        for (int i = 0; i < roomCount; i++)
        {
            dungeonRooms.add(createDungeonRoom());
        }

        return dungeonRooms;
    }

    private DungeonRoom createDungeonRoom()
    {
        return new DungeonRoom("Dungeon of " + getRandomName(), "",
getRandomItems(), random.nextInt(25), random.nextInt(25),
random.nextInt(100));
    }

    private String getRandomName()
    {
        return names.get(random.nextInt(names.size()));
    }

    private List<Item> getRandomItems()
    {
        List<Item> randomItems = new ArrayList<>();

        int itemCount = random.nextInt(3);

        for (int i = 0; i < itemCount; i++)
        {
            randomItems.add(items.get(random.nextInt(items.size())));
        }

        return randomItems;
    }

    public long getSeed()
    {
        return seed;
    }

    private void createNames()
    {
        // ...
    }
```

```
    private void createItems()
    {
        // ...
    }
```

2.1.4 `Dungeon`

`Dungeon` simulates the player going through the `DungeonRoom`s and creates a `DungeonRun` log record.

```
public class Dungeon
{
    private DungeonPlayer dungeonPlayer;
    private List<DungeonRoom> dungeonRooms;

    public Dungeon(long seed, int roomCount)
    {
        createNewDungeon(seed, roomCount);
    }

    public void createNewDungeon(long seed, int roomCount)
    {
        DungeonGenerator dungeonGenerator = new
DungeonGenerator(roomCount, seed);

        dungeonPlayer = dungeonGenerator.getDungeonPlayer();
        dungeonRooms = dungeonGenerator.getDungeonRooms();
    }

    public DungeonRun runDungeon()
    {
        for (DungeonRoom dungeonRoom : dungeonRooms)
        {
            runDungeonRoom(dungeonRoom);

            if (dungeonPlayer.isDead())
            {
                break;
            }
        }

        return new DungeonRun(  dungeonPlayer.getName(),
                                dungeonPlayer.getGold(),
                                dungeonPlayer.getRoomsVisited(),
                                !dungeonPlayer.isDead(),
                                dungeonPlayer.getItems());
    }
```

```
    private void runDungeonRoom(DungeonRoom dungeonRoom)
    {
        dungeonPlayer.roomVisited();
        dungeonPlayer.addGold(dungeonRoom.getGold());
        dungeonPlayer.addItems(dungeonRoom.getItems());
        dungeonPlayer.damage(dungeonRoom.getRoomDamage());
        dungeonPlayer.heal(dungeonRoom.getRoomHealth());
    }
}
```

2.1.5 `DungeonRun`

`DungeonRun` stores a run.

```
public record DungeonRun(String name, int goldAcquired, int roomsVisited,
boolean survived, List<Item> itemsCollected)
{
    // ...
}
```

2.1.6 `DungeonRunner`

Simple class with a static method that runs dungeons and produces a list of `DungeonRun`s.

```
public class DungeonRunner
{
    public static List<DungeonRun> runDungeons(int count, long seed)
    {
        List<DungeonRun> runs = new ArrayList<DungeonRun>();
        Random rand = new Random(seed);

        for (int i = 0; i < count; i++)
        {
            Dungeon dungeon = new Dungeon(rand.nextLong(Long.MAX_VALUE),
rand.nextInt(count * 10));
            runs.add(dungeon.runDungeon());
        }

        return runs;
    }
}
```

2.1.7 Test Output

```
List<DungeonRun> runs = DungeonRunner.runDungeons(20, 42L);

for (DungeonRun dungeonRun : runs)
```

```
{
    IO.println(dungeonRun.asShortString());
}
```

```
// for brevity, I printed the shortened versions here.
[DEAD] Glumble Snortwick | Rooms: 45 | Gold: 2455 | Items: 46 | Perished
[DEAD] Blatherskite McGurp | Rooms: 80 | Gold: 4276 | Items: 76 | Perished
[ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived
[ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 | Survived
[ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84 | Survived
```

2.2 Loop-based queries

```
static List<DungeonRun> runs = DungeonRunner.runDungeons(20, 42L);
```

2.2.1 Survivors

```
static List<DungeonRun> filterSurvivors()
{
    List<DungeonRun> survivors = new ArrayList<>();

    for (DungeonRun dungeonRun : runs)
    {
        if (dungeonRun.survived())
        {
            survivors.add(dungeonRun);
        }
    }

    return survivors;
}
```

```
IO.println("=== SURVIVORS ===");
for (DungeonRun run : filterSurvivors())
    IO.println(run.asShortString());
```

```
=== SURVIVORS ===
[ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived
[ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 | Survived
[ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84 | Survived
[ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
[ALIVE] Frizzle Norbax | Rooms: 0 | Gold: 0 | Items: 0 | Survived
[ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived
[ALIVE] Klorp Fibblenose | Rooms: 46 | Gold: 2271 | Items: 53 | Survived
[ALIVE] Throbble Dungsworth | Rooms: 30 | Gold: 1444 | Items: 31 |
```

```
Survived
[ALIVE] Snarfle Zun | Rooms: 113 | Gold: 5286 | Items: 130 | Survived
[ALIVE] Gorple Snagworth | Rooms: 65 | Gold: 3349 | Items: 69 | Survived
[ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived
[ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 | Survived
[ALIVE] Glorbnor | Rooms: 3 | Gold: 153 | Items: 0 | Survived
```

2.2.2 Top N Runs

I feel like sorting the array is kinda ruining the point here, but well, im kinda tired.

```
static List<DungeonRun> topRuns(int n)
{
    List<DungeonRun> topRuns = new ArrayList<>();
    List<DungeonRun> sorted = sortRunsByGoldDesc();

    for (int i = 0; i < n; i++)
    {
        topRuns.add(sorted.get(i));
    }

    return topRuns;
}

static List<DungeonRun> sortRunsByGoldDesc()
{
    List<DungeonRun> sortedRuns = new ArrayList<>(runs);

    for (int i = 0; i < sortedRuns.size() - 1; i++)
    {
        for (int j = 0; j < sortedRuns.size() - i - 1; j++)
        {
            if (sortedRuns.get(j).goldAcquired() < sortedRuns.get(j +
1).goldAcquired())
            {
                DungeonRun temp = sortedRuns.get(j);
                sortedRuns.set(j, sortedRuns.get(j + 1));
                sortedRuns.set(j + 1, temp);
            }
        }
    }

    return sortedRuns;
}
```

```
IO.println("\n=== TOP 5 RUNS BY GOLD ===");
for (DungeonRun run : topRuns(5))
```

```
    IO.println(run.asShortString());
```

```
=== TOP 5 RUNS BY GOLD ===
[ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
[ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 | Survived
[ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived
[ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived
[ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived
```

2.2.3 Total Gold

```
static int totalGold()
{
    int gold = 0;

    for (DungeonRun dungeonRun : runs)
    {
        gold += dungeonRun.goldAcquired();
    }

    return gold;
}
```

```
IO.println("\n=== TOTAL GOLD ACROSS ALL RUNS ===");
IO.println(totalGold() + "g");
```

```
=== TOTAL GOLD ACROSS ALL RUNS ===
74326g
```

2.2.4 Grouped Runs

Uses a HashMap to map player names to dungeon runs.

```
static HashMap<String, List<DungeonRun>> groupByPlayers()
{
    HashMap<String, List<DungeonRun>> playerRuns = new HashMap<>();

    for (DungeonRun dungeonRun : runs)
    {
        if (playerRuns.containsKey(dungeonRun.name()))
        {
            playerRuns.get(dungeonRun.name()).add(dungeonRun);
        }
        else
        {
```

```
            playerRuns.put(dungeonRun.name(), new ArrayList<>());
            playerRuns.get(dungeonRun.name()).add(dungeonRun);
        }
    }

    return playerRuns;
}
```

```
IO.println("\n=== RUNS BY PLAYER ===");
for (Map.Entry<String, List<DungeonRun>> entry :
groupByPlayers().entrySet())
{
    IO.println("\n~ " + entry.getKey() + " ~");
    for (DungeonRun run : entry.getValue())
        IO.println("  " + run.asShortString());
}
```

```
=== RUNS BY PLAYER ===
~ Glorbnor ~
  [ALIVE] Glorbnor | Rooms: 3 | Gold: 153 | Items: 0 | Survived

~ Throbble Dungsworth ~
  [ALIVE] Throbble Dungsworth | Rooms: 30 | Gold: 1444 | Items: 31 |
Survived

~ Snarfle Zun ~
  [ALIVE] Snarfle Zun | Rooms: 113 | Gold: 5286 | Items: 130 | Survived

~ Snaggle Pfunk ~
  [DEAD] Snaggle Pfunk | Rooms: 83 | Gold: 3789 | Items: 81 | Perished

~ Glumble Snortwick ~
  [DEAD] Glumble Snortwick | Rooms: 45 | Gold: 2455 | Items: 46 | Perished

~ Klorp Fibblenose ~
  [ALIVE] Klorp Fibblenose | Rooms: 46 | Gold: 2271 | Items: 53 | Survived

~ Mina Gaia ~
  [ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived

~ Blatherskite McGurp ~
  [DEAD] Blatherskite McGurp | Rooms: 80 | Gold: 4276 | Items: 76 |
Perished
  [DEAD] Blatherskite McGurp | Rooms: 23 | Gold: 1266 | Items: 20 |
Perished

~ Gobblewump ~
  [DEAD] Gobblewump | Rooms: 107 | Gold: 5209 | Items: 122 | Perished
```

```
~ Frizzle Norbax ~
  [ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
  [ALIVE] Frizzle Norbax | Rooms: 0 | Gold: 0 | Items: 0 | Survived

~ Splrx ~
  [DEAD] Splrx | Rooms: 27 | Gold: 1546 | Items: 19 | Perished

~ Gorple Snagworth ~
  [ALIVE] Gorple Snagworth | Rooms: 65 | Gold: 3349 | Items: 69 | Survived

~ Gobsnort Wumblemore ~
  [DEAD] Gobsnort Wumblemore | Rooms: 17 | Gold: 828 | Items: 9 | Perished

~ Winklenob ~
  [ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 | Survived

~ Angus McFife ~
  [ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84 | Survived

~ Thrumblewick ~
  [ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived

~ Grumplethwaite ~
  [ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived

~ Voorp Snibbleton ~
  [ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 |
Survived
```

2.3 Stream rewrites and advanced queries

2.3.1 Survivors

```
static List<DungeonRun> filterSurvivors()
{
    return runs.stream().filter(DungeonRun::survived).toList();
}
```

```
IO.println("=== SURVIVORS ===");
for (DungeonRun run : filterSurvivors())
    IO.println(run.asShortString());
```

```
=== SURVIVORS ===
[ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived
[ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 | Survived
```

```
[ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84 | Survived
[ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
[ALIVE] Frizzle Norbax | Rooms: 0 | Gold: 0 | Items: 0 | Survived
[ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived
[ALIVE] Klorp Fibblenose | Rooms: 46 | Gold: 2271 | Items: 53 | Survived
[ALIVE] Throbble Dungsworth | Rooms: 30 | Gold: 1444 | Items: 31 |
Survived
[ALIVE] Snarfle Zun | Rooms: 113 | Gold: 5286 | Items: 130 | Survived
[ALIVE] Gorple Snagworth | Rooms: 65 | Gold: 3349 | Items: 69 | Survived
[ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived
[ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 | Survived
[ALIVE] Glorbnor | Rooms: 3 | Gold: 153 | Items: 0 | Survived
```

2.3.2 Top 5 Players

```
static List<DungeonRun> topRuns(int n)
{
    return
runs.stream().sorted(Comparator.comparingInt(DungeonRun::goldAcquired)).to
List().reversed().subList(0, n);
}
```

```
IO.println("\n=== TOP 5 RUNS BY GOLD ===");
for (DungeonRun run : topRuns(5))
    IO.println(run.asShortString());
```

```
=== TOP 5 RUNS BY GOLD ===
[ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
[ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 | Survived
[ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived
[ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived
[ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived
```

2.3.3 Total Gold

```
static int totalGold()
{
    return runs.stream().mapToInt(DungeonRun::goldAcquired).sum();
}
```

```
IO.println("\n=== TOTAL GOLD ACROSS ALL RUNS ===");
IO.println(totalGold() + "g");
```

```
=== TOTAL GOLD ACROSS ALL RUNS ===
74326g
```

2.3.4 Runs by Player

```
static Map<String, List<DungeonRun>> groupByPlayers()
{
    return runs.stream().collect(Collectors.groupingBy(DungeonRun::name));
}
```

```
IO.println("\n=== RUNS BY PLAYER ===");
for (Map.Entry<String, List<DungeonRun>> entry :
groupByPlayers().entrySet())
{
    IO.println("\n~ " + entry.getKey() + " ~");
    for (DungeonRun run : entry.getValue())
        IO.println("  " + run.asShortString());
}
```

```
=== RUNS BY PLAYER ===

~ Glorbnor ~
  [ALIVE] Glorbnor | Rooms: 3 | Gold: 153 | Items: 0 | Survived

~ Snaggle Pfunk ~
  [DEAD] Snaggle Pfunk | Rooms: 83 | Gold: 3789 | Items: 81 | Perished

~ Snarfle Zun ~
  [ALIVE] Snarfle Zun | Rooms: 113 | Gold: 5286 | Items: 130 | Survived

~ Throbble Dungsworth ~
  [ALIVE] Throbble Dungsworth | Rooms: 30 | Gold: 1444 | Items: 31 |
Survived

~ Glumble Snortwick ~
  [DEAD] Glumble Snortwick | Rooms: 45 | Gold: 2455 | Items: 46 | Perished

~ Mina Gaia ~
  [ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived

~ Klorp Fibblenose ~
  [ALIVE] Klorp Fibblenose | Rooms: 46 | Gold: 2271 | Items: 53 | Survived

~ Gobblewump ~
  [DEAD] Gobblewump | Rooms: 107 | Gold: 5209 | Items: 122 | Perished
```

```
~ Blatherskite McGurp ~
  [DEAD] Blatherskite McGurp | Rooms: 80 | Gold: 4276 | Items: 76 |
Perished
  [DEAD] Blatherskite McGurp | Rooms: 23 | Gold: 1266 | Items: 20 |
Perished

~ Frizzle Norbax ~
  [ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
  [ALIVE] Frizzle Norbax | Rooms: 0 | Gold: 0 | Items: 0 | Survived

~ Splrx ~
  [DEAD] Splrx | Rooms: 27 | Gold: 1546 | Items: 19 | Perished

~ Gorple Snagworth ~
  [ALIVE] Gorple Snagworth | Rooms: 65 | Gold: 3349 | Items: 69 | Survived

~ Gobsnort Wumblemore ~
  [DEAD] Gobsnort Wumblemore | Rooms: 17 | Gold: 828 | Items: 9 | Perished

~ Angus McFife ~
  [ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84 | Survived

~ Winklenob ~
  [ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 | Survived

~ Thrumblewick ~
  [ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived

~ Grumplethwaite ~
  [ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived

~ Voorp Snibbleton ~
  [ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 |
Survived
```

2.3.5 Best Run per Player

```
static Map<String, DungeonRun> bestRunPerPlayer()
{
    Map<String, DungeonRun> bestRunPerPlayer = new HashMap<>();

    groupByPlayers()
            .forEach( (key, value) -> bestRunPerPlayer.put(
                    key,

value.stream().sorted(Comparator.comparingInt(DungeonRun::goldAcquired)
                    ).toList().getLast()));
```

```
    return bestRunPerPlayer;
}
```

```
IO.println("\n=== BEST RUN PER PLAYER ===");
for (Map.Entry<String, DungeonRun> entry : bestRunPerPlayer().entrySet())
    IO.println(entry.getKey() + " → " +
entry.getValue().asShortString());
```

```
=== BEST RUN PER PLAYER ===
Glorbnor → [ALIVE] Glorbnor | Rooms: 3 | Gold: 153 | Items: 0 | Survived
Snaggle Pfunk → [DEAD] Snaggle Pfunk | Rooms: 83 | Gold: 3789 | Items: 81
| Perished
Snarfle Zun → [ALIVE] Snarfle Zun | Rooms: 113 | Gold: 5286 | Items: 130
| Survived
Throbble Dungsworth → [ALIVE] Throbble Dungsworth | Rooms: 30 | Gold:
1444 | Items: 31 | Survived
Glumble Snortwick → [DEAD] Glumble Snortwick | Rooms: 45 | Gold: 2455 |
Items: 46 | Perished
Mina Gaia → [ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 |
Survived
Klorp Fibblenose → [ALIVE] Klorp Fibblenose | Rooms: 46 | Gold: 2271 |
Items: 53 | Survived
Gobblewump → [DEAD] Gobblewump | Rooms: 107 | Gold: 5209 | Items: 122 |
Perished
Blatherskite McGurp → [DEAD] Blatherskite McGurp | Rooms: 80 | Gold: 4276
| Items: 76 | Perished
Frizzle Norbax → [ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 |
Items: 172 | Survived
Splrx → [DEAD] Splrx | Rooms: 27 | Gold: 1546 | Items: 19 | Perished
Gorple Snagworth → [ALIVE] Gorple Snagworth | Rooms: 65 | Gold: 3349 |
Items: 69 | Survived
Gobsnort Wumblemore → [DEAD] Gobsnort Wumblemore | Rooms: 17 | Gold: 828
| Items: 9 | Perished
Angus McFife → [ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84
| Survived
Winklenob → [ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 |
Survived
Thrumblewick → [ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items:
112 | Survived
Grumplethwaite → [ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 |
Items: 107 | Survived
Voorp Snibbleton → [ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 |
Items: 150 | Survived
```

2.3.6 Run Count

Why does a `TreeMap` fit better than a `HashMap`? The tree automatically sorts when inserting an element, where as `HashMap`s aren't sorted at all.

```
static TreeMap<String, Integer> runCountPerPlayer()
{
    TreeMap<String, Integer> runCountPerPlayer = new TreeMap<>();

    groupByPlayers()
            .forEach((key, value) -> runCountPerPlayer.put(key,
value.size()));

    return runCountPerPlayer;
}
```

```
IO.println("\n=== RUN COUNT PER PLAYER ===");
for (Map.Entry<String, Integer> entry : runCountPerPlayer().entrySet())
    IO.println(entry.getKey() + " -> " + entry.getValue() + " runs");
```

```
=== RUN COUNT PER PLAYER ===
Angus McFife -> 1 runs
Blatherskite McGurp -> 2 runs
Frizzle Norbax -> 2 runs
Glorbnor -> 1 runs
Glumble Snortwick -> 1 runs
Gobblewump -> 1 runs
Gobsnort Wumblemore -> 1 runs
Gorple Snagworth -> 1 runs
Grumplethwaite -> 1 runs
Klorp Fibblenose -> 1 runs
Mina Gaia -> 1 runs
Snaggle Pfunk -> 1 runs
Snarfle Zun -> 1 runs
Splrx -> 1 runs
Throbble Dungsworth -> 1 runs
Thrumblewick -> 1 runs
Voorp Snibbleton -> 1 runs
Winklenob -> 1 runs
```

2.3.7 Average Gold

```
static int averageGold()
{
    return runs.stream().mapToInt(DungeonRun::goldAcquired).sum() /
runs.size();
}
```

```
IO.println("\n=== AVERAGE GOLD PER RUN ===");
IO.println(averageGold() + "g");
```

```
=== AVERAGE GOLD PER RUN ===
3716g
```

2.3.8 Raw Output

B Queries:

```
C:\Users\cmdrp\.jdks\openjdk-25\bin\java.exe "-
javaagent:C:\Users\cmdrp\AppData\Local\Programs\IntelliJ IDEA
Ultimate\lib\idea_rt.jar=49767" -Dfile.encoding=UTF-8 -
Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath
"R:\GitHub\Zoes-Hagenberg-2-Electric-Boogaloo\Algorithmic
Thinking\tasks\A04 - Trees and Maps\TreesAndMaps\target\classes"
gay.fox.dungeon.query.DungeonQueries_B
=== SURVIVORS ===
[ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived
[ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 | Survived
[ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84 | Survived
[ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
[ALIVE] Frizzle Norbax | Rooms: 0 | Gold: 0 | Items: 0 | Survived
[ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived
[ALIVE] Klorp Fibblenose | Rooms: 46 | Gold: 2271 | Items: 53 | Survived
[ALIVE] Throbble Dungsworth | Rooms: 30 | Gold: 1444 | Items: 31 |
Survived
[ALIVE] Snarfle Zun | Rooms: 113 | Gold: 5286 | Items: 130 | Survived
[ALIVE] Gorple Snagworth | Rooms: 65 | Gold: 3349 | Items: 69 | Survived
[ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived
[ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 | Survived
[ALIVE] Glorbnor | Rooms: 3 | Gold: 153 | Items: 0 | Survived

=== TOP 5 RUNS BY GOLD ===
[ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
[ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 | Survived
[ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived
[ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived
[ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived

=== TOTAL GOLD ACROSS ALL RUNS ===
74326g

=== RUNS BY PLAYER ===

~ Glorbnor ~
  [ALIVE] Glorbnor | Rooms: 3 | Gold: 153 | Items: 0 | Survived
```

~ Throbble Dungsworth ~
  [ALIVE] Throbble Dungsworth | Rooms: 30 | Gold: 1444 | Items: 31 | Survived

~ Snarfle Zun ~
  [ALIVE] Snarfle Zun | Rooms: 113 | Gold: 5286 | Items: 130 | Survived

~ Snaggle Pfunk ~
  [DEAD] Snaggle Pfunk | Rooms: 83 | Gold: 3789 | Items: 81 | Perished

~ Glumble Snortwick ~
  [DEAD] Glumble Snortwick | Rooms: 45 | Gold: 2455 | Items: 46 | Perished

~ Klorp Fibblenose ~
  [ALIVE] Klorp Fibblenose | Rooms: 46 | Gold: 2271 | Items: 53 | Survived

~ Mina Gaia ~
  [ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived

~ Blatherskite McGurp ~
  [DEAD] Blatherskite McGurp | Rooms: 80 | Gold: 4276 | Items: 76 | Perished
  [DEAD] Blatherskite McGurp | Rooms: 23 | Gold: 1266 | Items: 20 | Perished

~ Gobblewump ~
  [DEAD] Gobblewump | Rooms: 107 | Gold: 5209 | Items: 122 | Perished

~ Frizzle Norbax ~
  [ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
  [ALIVE] Frizzle Norbax | Rooms: 0 | Gold: 0 | Items: 0 | Survived

~ Splrx ~
  [DEAD] Splrx | Rooms: 27 | Gold: 1546 | Items: 19 | Perished

~ Gorple Snagworth ~
  [ALIVE] Gorple Snagworth | Rooms: 65 | Gold: 3349 | Items: 69 | Survived

~ Gobsnort Wumblemore ~
  [DEAD] Gobsnort Wumblemore | Rooms: 17 | Gold: 828 | Items: 9 | Perished

~ Winklenob ~
  [ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 | Survived

~ Angus McFife ~
  [ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84 | Survived

~ Thrumblewick ~
  [ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived

```
~ Grumplethwaite ~
  [ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived

~ Voorp Snibbleton ~
  [ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 |
Survived

Process finished with exit code 0
```

C Queries:

```
=== SURVIVORS ===
[ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived
[ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 | Survived
[ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84 | Survived
[ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
[ALIVE] Frizzle Norbax | Rooms: 0 | Gold: 0 | Items: 0 | Survived
[ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived
[ALIVE] Klorp Fibblenose | Rooms: 46 | Gold: 2271 | Items: 53 | Survived
[ALIVE] Throbble Dungsworth | Rooms: 30 | Gold: 1444 | Items: 31 |
Survived
[ALIVE] Snarfle Zun | Rooms: 113 | Gold: 5286 | Items: 130 | Survived
[ALIVE] Gorple Snagworth | Rooms: 65 | Gold: 3349 | Items: 69 | Survived
[ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived
[ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 | Survived
[ALIVE] Glorbnor | Rooms: 3 | Gold: 153 | Items: 0 | Survived

=== TOP 5 RUNS BY GOLD ===
[ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
[ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 | Survived
[ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived
[ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived
[ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived

=== TOTAL GOLD ACROSS ALL RUNS ===
74326g

=== RUNS BY PLAYER ===

~ Glorbnor ~
  [ALIVE] Glorbnor | Rooms: 3 | Gold: 153 | Items: 0 | Survived

~ Snaggle Pfunk ~
  [DEAD] Snaggle Pfunk | Rooms: 83 | Gold: 3789 | Items: 81 | Perished

~ Snarfle Zun ~
  [ALIVE] Snarfle Zun | Rooms: 113 | Gold: 5286 | Items: 130 | Survived
```

```
~ Throbble Dungsworth ~
  [ALIVE] Throbble Dungsworth | Rooms: 30 | Gold: 1444 | Items: 31 |
Survived

~ Glumble Snortwick ~
  [DEAD] Glumble Snortwick | Rooms: 45 | Gold: 2455 | Items: 46 | Perished

~ Mina Gaia ~
  [ALIVE] Mina Gaia | Rooms: 135 | Gold: 6242 | Items: 143 | Survived

~ Klorp Fibblenose ~
  [ALIVE] Klorp Fibblenose | Rooms: 46 | Gold: 2271 | Items: 53 | Survived

~ Gobblewump ~
  [DEAD] Gobblewump | Rooms: 107 | Gold: 5209 | Items: 122 | Perished

~ Blatherskite McGurp ~
  [DEAD] Blatherskite McGurp | Rooms: 80 | Gold: 4276 | Items: 76 |
Perished
  [DEAD] Blatherskite McGurp | Rooms: 23 | Gold: 1266 | Items: 20 |
Perished

~ Frizzle Norbax ~
  [ALIVE] Frizzle Norbax | Rooms: 170 | Gold: 8144 | Items: 172 | Survived
  [ALIVE] Frizzle Norbax | Rooms: 0 | Gold: 0 | Items: 0 | Survived

~ Splrx ~
  [DEAD] Splrx | Rooms: 27 | Gold: 1546 | Items: 19 | Perished

~ Gorple Snagworth ~
  [ALIVE] Gorple Snagworth | Rooms: 65 | Gold: 3349 | Items: 69 | Survived

~ Gobsnort Wumblemore ~
  [DEAD] Gobsnort Wumblemore | Rooms: 17 | Gold: 828 | Items: 9 | Perished

~ Angus McFife ~
  [ALIVE] Angus McFife | Rooms: 76 | Gold: 3083 | Items: 84 | Survived

~ Winklenob ~
  [ALIVE] Winklenob | Rooms: 102 | Gold: 5291 | Items: 112 | Survived

~ Thrumblewick ~
  [ALIVE] Thrumblewick | Rooms: 113 | Gold: 6108 | Items: 112 | Survived

~ Grumplethwaite ~
  [ALIVE] Grumplethwaite | Rooms: 119 | Gold: 5923 | Items: 107 | Survived

~ Voorp Snibbleton ~
```

```
  [ALIVE] Voorp Snibbleton | Rooms: 156 | Gold: 7663 | Items: 150 |
Survived
```

Only differences was due to the order, because I wasn't using a `LinkedHashMap`.